

Supplementary Material: Indexing Made Consistent and Efficient on Partially Cache Coherent Shared Memory with SPORE Guidelines

A Converting CCIndex to PCCIndex

This section shows the detailed code of how we convert several commonly used CCIndexes to PCCIndexes.

In the following, we show the detailed code and correctness discussion of:

- BTreeOLC, ARTOLC, Masstree in Optimistic Lock-based Index (§A.1)
- BwTree and ClevelHash in Lock-free Index (§A.2)
- CLHT and HOT in ROWEX-based Index (§A.3)

A.1 Optimistic Lock-based Index

BTreeOLC [4], ARTOLC [4], and Masstree [5] are three optimistic lock-based indexes. They all use the optimistic lock to coordinate concurrent readers and writers. Fig. 1 shows the detailed code of the optimistic lock part. This optimistic lock part is shared by BTreeOLC [4], ARTOLC [4] and Masstree [5]. The code we add is in the boxes, which is used to ensure consistency of optimistic lock-based indexes on PCC platforms.

Sync-data. The sync-data of optimistic lock-based indexes is the 64-bit *version* field (typeVersionLockObsolete in Fig. 1), which encodes a 62-bit version counter, a 1-bit lock bit, and a 1-bit obsolete flag. The obsolete flag indicates whether the node is deleted.

Protected-data. The protected-data of BTreeOLC/ARTOLC/Masstree is the tree's node. Reading or writing the node's data is protected by the lock and version check.

Linearizability. To ensure linearizability of optimistic lock-based indexes without relying on hardware cache coherence, we change the type of the lock's *version* field from `std::atomic<uint64_t>` to our `pcc::nt` pointer type (code block ①). `pcc::nt` type is a custom class that provides `pLoad`, `pStore` and `pCAS` operations. This substitution leaves all lock-related call sites unchanged, while transparently replacing the original `Load/Store/CAS` operations on the *version* field with their `pLoad/pStore/pCAS` counterparts. For example, the original `compare_exchange_strong` on *version* is now executed via `pCAS`, making each successful lock acquisition or release a cache-bypassing atomic update. As a result, updates to *version* (i.e., the sync-data of the lock) become immediately visible to all hosts, which use `pLoad` to access the *version* field, thereby preserving the linearizability of the lock protocol on PCC platforms.

In addition, we must ensure that the protected data (i.e., the node's contents) is flushed before reading and after writing. To achieve this, we introduce a per-node-type `flushNode` function. Concretely, `flushNode` issues a cacheline flush operation (`clflush`) followed by a memory fence operation (`mfence`) to write-back the node's cachelines to memory. We invoke `flushNode` after acquiring the read lock (③) or the write lock (④) and before reading the node's contents. We also invoke `flushNode` after writing the actual node data and before releasing the write lock (⑥).

Durable linearizability. We guarantee durable linearizability by adapting techniques proposed in RECIPE [3].

Failure isolation. To ensure failure isolation, we implement the lock as a *recoverable lock*. Each lock embeds the owning host's identifier (*hostId*) into the lock state, so that recovery procedures can determine the lock holder upon recovery. Specifically, the *recoverable lock* encodes a 16-bit *hostId* in the 64-bit *version* field. When trying to get the write lock, the writer also needs to encode its *hostId* into the *version* field with `setHostId` (code block ⑤), and use a single CAS to ensure that the *hostId* is set together with acquiring the lock. When releasing the write lock, the writer also needs to clear the *hostId*, and use a single `pStore` to change the *version* field (code block ⑦).

Host liveness is monitored by a controller following the approach used in Lupin [6]: every host periodically writes a heartbeat message into shared memory, enabling the controller to detect whether a host has failed. Once a thread repeatedly times out during acquisition of a lock, it consults the controller to check whether the lock owner has failed; if the owner has failed, the controller releases the lock, allowing others to proceed.

We assume that the controller is implemented on the memory controller of CXL/UB systems; and we do not consider failures of the controller itself.

A writer that crashes and later recovers may continue using its original *hostId*, but it must notify the controller before resuming execution. This ensures that the controller will no longer clear locks associated with that writer, preventing situations where a lock held by the writer is erroneously released by the controller during recovery.

Our current recoverable-lock design tracks liveness only at the host level. Supporting process-level ID tracking would require the controller to detect the liveness of individual

```

1 struct OptLock {
2     #ifdef PCC_PLATFORM
3         pcc::nt_uint64_t typeVersionLockObsolete{0b100};
4         virtual void flush() = 0;
5         #define LOCK_BIT (0b10 << 16)
6     #else
7         std::atomic_uint64_t typeVersionLockObsolete{0b100};
8         #define LOCK_BIT (0b10)
9     #endif
10
11     bool isLocked(uint64_t version) {
12         return (version & LOCK_BIT) == LOCK_BIT;
13     }
14
15     bool isObsolete(uint64_t version) { return (version & 0b1) == 0b1; }
16
17     uint64_t clearHostId(uint64_t version) { return (version & ~(0xFFFEE)); }
18
19     uint64_t setHostId(uint64_t version, uint16_t hostId) {
20         return clearHostId(version) | (hostId << 1);
21     }
22
23     uint64_t readLockOrRestart(bool &needRestart, bool needFlush = true) {
24         uint64_t version = typeVersionLockObsolete.load();
25         if (isLocked(version) || isObsolete(version)) {
26             _mm_pause();
27             needRestart = true;
28         }
29     #ifdef PCC_PLATFORM
30         /* Flush the node if lock is acquired */
31         if (!needRestart && needFlush) flushNode();
32     #endif
33     }
34     return version;
35 }
36
37 void writeLockOrRestart(uint64_t hostId, uint16_t hostId,
38     bool &needRestart) {
39     uint64_t version = readLockOrRestart(needRestart, false);
40     if (needRestart) return;
41
42     upgradeToWriteLockOrRestart(version, needRestart);
43     if (needRestart) return;
44     #ifdef PCC_PLATFORM
45     else flushNode();
46     #endif
47 }
48
49 void upgradeToWriteLockOrRestart(uint64_t &version, bool &needRestart) {
50     if (typeVersionLockObsolete.compare_exchange_strong(version,
51     #ifdef PCC_PLATFORM
52         setHostId(version, hostId) + LOCK_BIT)) {
53     #else
54         version + LOCK_BIT)) {
55     #endif
56         version = version + LOCK_BIT;
57     } else {
58         _mm_pause();
59         needRestart = true;
60     }
61 }
62
63 void writeUnlock(bool needFlush=false) {
64     #ifdef PCC_PLATFORM
65     /* Only flush the node if node is modified */
66     if (needFlush) flushNode();
67     #endif
68
69     typeVersionLockObsolete.store(
70     #ifdef PCC_PLATFORM
71         clearHostId(typeVersionLockObsolete.load()) + LOCK_BIT;
72     #else
73         typeVersionLockObsolete.load() + LOCK_BIT;
74     #endif
75 );
76 }
77
78 struct NodeBase : public OptLock {
79     PageType type; /* LeafNode or InnerNode */
80     uint16_t count;
81     /* Impl flush method according to Node type */
82 };

```

Fig. 1. C++ Code of converted optimistic lock part shared by BTreeOLC, ARTOLC and Masstree.

processes, as well as assigning globally unique process identifiers across the entire CXL/UB cluster.

```

1 class Index {
2
3     #ifdef PCC_PLATFORM
4         pcc::nt<NodeID> root_id;
5         std::array<pcc::nt<BaseNode*>, MAPPING_TABLE_SIZE> mapping_table;
6     #else
7         std::atomic<NodeID> root_id;
8         std::array<std::atomic<BaseNode*>, MAPPING_TABLE_SIZE> mapping_table;
9     #endif
10     ...
11
12     inline bool InstallNodeToReplace(NodeID node_id, const BaseNode *node_p,
13         const BaseNode *prev_p) {
14         #ifdef PCC_PLATFORM
15         node_p->flushNode();
16         #endif
17         return mapping_table[node_id].compare_exchange_strong(prev_p, node_p);
18     }
19
20     inline void InstallNewNode(NodeID node_id, const BaseNode *node_p) {
21         #ifdef PCC_PLATFORM
22         node_p->flushNode();
23         #endif
24         mapping_table[node_id].store(node_p);
25         return;
26     }
27
28     inline const BaseNode *GetNode(const NodeID node_id) {
29         return mapping_table[node_id].load();
30     }
31 };

```

Fig. 2. C++ Code of converted BwTree, a lock-free index

Even with recoverable locks, operations may still block until the controller clears the lock, incurring non-trivial performance overhead. For this reason, our case studies focus on porting lock-free data structures.

A.2 Lock-free Index

We use BwTree as an example to show how we extend a lock-free index to work correctly on PCC platforms. The C++ code and our modifications are shown in Fig. 2.

Linearizability (R1). To ensure linearizability of lock-free indexes, we replace the `std::atomic root_id` and pointer in the mapping table (`mapping_table[node_id]`) with our `pcc::nt` pointer (code block ①). With `pcc::nt`, installing nodes or accessing nodes from the mapping table will be replaced by cache-bypassing operations. Besides, we also need to flush the newly allocated node with `clwb + mfence` as shown in code block ② and ③.

Durable linearizability (R2.1). RECIPE already proposes a converted BwTree that is durable linearizable, and we follow their method.

Failure isolated (R2.2). Since BwTree is a lock-free index, failure isolation is already guaranteed since no one will hold the lock.

A.3 ROWEX-based Index

We first give a general description of ROWEX-based index, followed by concrete code of how we convert CLHT [2], a ROWEX-based hash table.

```

LOCK(lock):
- while(!CAS(&lock, 0, 1));
+ while(!pCAS(&lock, 0, 1)); ①
UNLOCK(lock): /*pStore: cache-
- lock = 0;      bypassing store*/
+ pStore(&lock, 0); ②

LOOKUP(key):
/* Same as Lock-free Index */

INSERT(key, val):
/* Lock all affected nodes */
LOCK(node_p->lock); LOCK(...); ③

/* Same as Lock-free index */
new_node_p = malloc(NODE_SIZE);
... /* update new node */
+ clwb(new_node_p); mfence();
- CAS(&nodes[i], node_p, new_node_p);
+ pCAS(&nodes[i], node_p, new_node_p);

/* UnLock all affected nodes */
UNLOCK(node_p->lock); UNLOCK(...); ④
return;

```

Fig. 3. General code for converting ROWEX-based index

A.3.1 General description of ROWEX-based index. ROWEX (Read-Optimized Write EXclusion) [4] allows readers to run without locks or version checks by requiring writers to maintain a consistent index at all times.

Unlike lock-free index, concurrent writers must still be serialized by acquiring locks. We use HOT [1] and CLHT [2] as two examples:

- In HOT [1], which is a trie-based index that performs out-of-place node updates with a single CAS of the node pointer, two concurrent writers may each install a different new node, losing one of the nodes (e.g., one writer replaces a node N with N' while another replaces N 's child C with C' , C' might become unreachable).
- CLHT [2] also requires acquiring locks of the buckets that need to be modified, in order to prevent two concurrent writers of the same key (e.g., one writer finds an empty slot in a bucket and inserts $K = 1, V = 1$, while another writer finds the same empty slot and inserts $K = 1, V = 2$; the second writer will overwrite the first writer's insert). Thus, writers first acquire locks of all nodes that need to be modified (③ in Fig. 3), update the nodes, and release all locks after the update (④ in Fig. 3).

Sync-data. We identify the lock as the sync-data, and operate it with pCAS (①) and pStore (②). Note that pStore is sufficient because the lock is now held by one thread; thus, this thread can use an atomic pStore instead of pCAS to clear the lock bit.

Additionally, each ROWEX uses different atomic variables according to their implementation. We also identify these atomic variables as the sync-data, and operate them with pLoad, pStore or pCAS according to their original implementation. For example, in CLHT [2], the 8-byte key and value

are the atomic variables, which are used to obtain atomic snapshots of key-value pairs. In HOT [1], the atomic variable is the pointer to the node, which is used to conduct out-of-place node updates by a single CAS of the node pointer. Besides, HOT also requires atomic 8-byte Load/Store operations on values in leaf nodes, since these values are updated in-place with a single atomic pStore operation. We therefore treat these as sync-data and operate on them using pLoad and pStore (a cache-bypassing store).

A.3.2 A concrete example: CLHT [2]. We give detailed code together with our modifications in Fig. 4.

Linearizability (R1). To ensure linearizability of CLHT [2], we identify atomic variables as the *lock*, the *key* and *value*, and the pointer to the bucket. We declare these atomic variables as `pcc:nt` type. Then, Load, Store, and CAS operations on these atomic variables will be automatically replaced by cache-bypassing operations.

The key and value are operated with atomic instructions to obtain atomic snapshots of key-value pairs. Specifically, the Lookup (`clht_get` in Fig. 4) operation finds the key/value by first loading the value, then loading the key, and loading value again and checking if the value is changed (code snippet ② in Fig. 4). Since the Insert (`clht_put` in Fig. 4) operation always updates value first then updates the key, Lookup operation will always get a key/value pair that belongs to a consistent snapshot.

Since all data are operated with atomic instructions, there is no protected-data, and thus we do not need to flush the data to memory.

Crash consistency (R2.1). The crash consistency is guaranteed with techniques in RECIPE [3]. As presented by RECIPE, all updates to CLHT are conducted through a single atomic Store operation. Thus, a crash at any point leaves the index consistent, so crash recovery is not required. Unlike RECIPE, our pStore operation directly changes the key/value and other data in shared memory, ensuring that every store is durable. Thus, we do not need to invoke extra write-back operations as RECIPE [3] does (see `#ifdef PER-SIST` code below code snippet ③, which is where RECIPE [3] inserts cacheline write-back instructions).

Fault Isolation (R2.2). Since CLHT relies on write-lock to avoid write-write conflict, we should solve the blocking issue when a lock's owner fails while holding the lock. We use the recoverable lock similar to BTreeOLC in Fig. 1.

B Details on G2: Replicating Pointers

This section shows:

- § B.1: pseudo code of *LoadPtr* and *CASPtr* mechanism (Algorithm 1), which is used to load and CAS replicated pointers following our R-Guideline (G2).

```

1 typedef struct ALIGNED(CACHE_LINE_SIZE) bucket_s
2 {
3     clht_lock_t lock;
4     #ifdef PCC
5         nt<uint32_t> hops;
6         nt<clht_addr_t> key[ENTRIES_PER_BUCKET];
7         nt<clht_val_t> val[ENTRIES_PER_BUCKET];
8         nt<struct bucket_s*> next;
9     #else
10        uint32_t hops;
11        clht_addr_t key[ENTRIES_PER_BUCKET];
12        clht_val_t val[ENTRIES_PER_BUCKET];
13        struct bucket_s* next;
14    #endif
15 } bucket_t;
16
17 clht_val_t clht_get(clht_hashtable_t* hashtable, clht_addr_t key)
18 {
19     size_t bin = clht_hash(hashtable, key);
20     CLHT_GC_HT_VERSION_USED(hashtable);
21     bucket_t* bucket = hashtable->table + bin;
22
23     uint32_t j;
24     do
25     {
26         for (j = 0; j < ENTRIES_PER_BUCKET; j++)
27         {
28             clht_val_t val = bucket->val[j];
29             if (bucket->key[j] == key)
30             {
31                 if (likely(bucket->val[j] == val))
32                 {
33                     return val;
34                 }
35                 else
36                 {
37                     return 0;
38                 }
39             }
40             bucket = bucket->next;
41         }
42     } while (unlikely(bucket != NULL));
43     return 0;
44 }
45
46 bool clht_put(clht_t* h, clht_addr_t key, clht_val_t val)
47 {
48     clht_hashtable_t* hashtable = h->data.fields.ht;
49     size_t bin = clht_hash(hashtable, key);
50     bucket_t* bucket = hashtable->table + bin;
51
52     clht_lock_t *lock = &(bucket->lock);
53     while (!LOCK_ACQ(lock, hashtable))
54     {
55         ...
56     }
57
58     do {
59         ... // find a empty bucket
60     }
61     {
62         b->val[0] = val;
63         b->key[0] = key;
64     }
65     #ifdef PERSIST // Code original present in RECIPE
66     // clwb((char *)b, sizeof(bucket_t));
67     #endif
68     LOCK_RLS(lock);
69     ... /* other logic */
70     return true;
71     bucket = bucket->next;
72 }
73 while (true);
74 }
75

```

Fig. 4. C++ Code of CLHT [2]’s data structure and Get/Put operation.

- §B.2: linearizability of G2 mechanism, together with a TLA+ verification.
- §B.3: crash consistency of G2 mechanism.
- §B.4: corner cases of G2 mechanism, e.g., crashed *CASPtr* or *LoadPtr*, another *CASPtr* during *HelpUpdate*, slow/preempted helpers, ABA problem, etc.

B.1 Pseudo Code of LoadPtr and CASPtr

Parameters. G is global pointer; $R[]$ is the array of pointer replicas; N is the total number of replicas.

CASPtr. *CASPtr* performs CAS on the global pointer and updates all replicas. It takes *old_val* and *new_val* as parameters, and returns true if the CAS is successful, and false otherwise.

Phase-1 (Line 13–15): *CASPtr* first pLoads the global pointer G (Line 12), and checks its lock bit (Line 14). If the lock bit is set, it means that the last-round update of G is still in progress, and *CASPtr* will first help finish the last-round update by calling *HelpUpdate* (Line 15). *expected* is set to G , and *start_id* is set to 0 to allow traversing all replicas.

Phase-2 (Line 16–18): After the last-round update is finished, *CASPtr* will pCAS G to *new_val*. The *new_val* will set the lock bit to 1, and using pCAS to update G can ensure the set of lock bit and compare-and-swap of G are atomic (Line 17). If CAS fails, it means that the G has been updated by other threads, and *CASPtr* will return false. Otherwise, *CASPtr* will enter the next phase.

Phase-3 (Line 19–21): If the CAS in *Phase-2* is successful, G is successfully updated to *new_val*, and *CASPtr* will help propagate the update to all replicas by calling *HelpUpdate* (Line 20). *expected* is set to *new_val*, and *start_id* is set to 0 to allow traversing all replicas. After the help update is finished, all replicas must have been updated to *new_val* (it is possible that G is updated to *new_val* and updated again at this time, but we can safely return true because the CAS has actually been finished). Besides, the help update phase will already clear the lock bit of G . Then, *CASPtr* can directly return *True*.

LoadPtr. *LoadPtr* takes the replica id as parameter, and returns the value of the pointer. We set the replica id as *thread_id mod replica_number*. Thus, the number of replicas does not need to exactly match the number of threads.

Case-1 (Line 4–6): If the lock bit of the replica is not set (Line 5 in Algorithm 1), it means that either (1) G is not being updated, or (2) G is being updated, and this replica is the *old_value*. In these two cases, *LoadPtr* can directly return the value of the replica (Line 6). This *LoadPtr* is considered linearized before the *CASPtr* of G .

Case-2 (Line 7–10): If the lock bit of the replica is set, it means that the replica is being updated, and the current replica equals to *new_val*. Then, *LoadPtr* will help update the replica to the value of G by calling *HelpUpdate* with

Algorithm 1: Help mechanism for updating replicas

```
1 # G: global pointer; R[]: pointer replicas; N: replica number
2 Function LoadPtr(id):
3   curR ← pLoad(R[id])
4   # Case1: unlocked R[id], directly return
5   if lock bit of curR is not set then
6     return curR
7   # Case2: locked R[id], help update and clear lock bit
8   HelpUpdate(ClearLock(curR), id + 1)
9   pCAS(R[id], curR, ClearLock(curR))
10  return ClearLock(curR)
11 Function CASPtr(old_val, new_val):
12  curG ← pLoad(G)
13  # Phase1: if G is locked, help last-round update of G
14  if lock bit of curG is set then
15    HelpUpdate(ClearLock(curG), 0)
16  # Phase2: CAS G from old_val to new_val
17  if ! pCAS(G, old_val, SetLock(new_val)) then
18    return false
19  # Phase3: update all replicas to G
20  HelpUpdate(new_val, 0)
21  return true
22 Function HelpUpdate(expected, start_id):
23  # Phase1: update each replica R[i] to expected
24  foreach replica R[i] in R[start_id : N - 1] do
25    curG ← pLoad(G)
26    # If lock bit of G is cleared, all replicas are updated to
    # expected, finish HelpUpdate
27    if ClearLock(curG) = expected ∧ lock bit of curG
    is not set then
28      return
29    r ← pLoad(R[i])
30    if ClearLock(r) ≠ expected then
31      curG ← pLoad(G)
32      # If G is changed, HelpUpdate finishes
33      if ClearLock(curG) ≠ expected then
34        return
35      # Update R[i] to expected
36      pCAS(R[i], r, SetLock(expected))
37      if pLoad(R[i]) ≠ expected then
38        # If R[i] is not updated to expected, which
        # means that G must have been changed,
        # and change R[i] again to a newer value
39        return
40  # Phase2: clear lock bit of G
41  pCAS(G, SetLock(expected), expected)
42  return
```

$expected = G$ and the $start_id = id + 1$ to traverse replicas after the current replica $R[i]$ (Line 8 in Algorithm 1). The start index is $id + 1$ because replicas are always updated from

$id = 0$ to $id = N - 1$, thus, a replica with $R[i] = new_val$ indicates that all replicas before it (i.e., $R[0]$ to $R[id - 1]$) have already been updated to new_val .

HelpUpdate. *HelpUpdate* is used to help update the replicas to the expected value. It takes the *expected* value and the *start_id* as parameters, and returns when the *expected* value has been updated. It can be called by concurrent threads. For example, many threads can together try to update the replicas to the same expected value, and once a thread succeeds, other threads will finish the *HelpUpdate* and return.

Phase-1 (Line 23–39): *HelpUpdate* first traverses replicas with index starting from *start_id* to $N - 1$ (i.e., $R[start_id]$ to $R[N - 1]$). For each $R[i]$, it checks whether the value of the replica equals *expected*; if not, it loads G again and updates $R[i]$ to *expected* (Line 24–39 in Algorithm 1).

In each round of updating $R[i]$ to *expected*, *HelpUpdate* will first pLoad G (Line 25), and check its lock bit (Line 27). If $G == expected$ and G 's lock bit is cleared, it means that all replicas have already been updated to *expected*. Therefore, *HelpUpdate* will directly return (Line 28).

If G is still locked, *HelpUpdate* will check the value of the replica $R[i]$ (Line 30). If G is not equal to *expected* (Line 33), it means that G has been updated to a newer value, and the *HelpUpdate* stops and returns. Otherwise, *HelpUpdate* will pCAS the replica $R[i]$ to *expected* (Line 36), and go to the next update round. If the pCAS fails, it means that the replica $R[i]$ has been updated by other threads, and *HelpUpdate* stops and returns (Line 39). Otherwise, *HelpUpdate* will continue checking the next replica.

If all replicas have been checked, all replicas are updated to *expected*, and *HelpUpdate* will enter the next phase.

Phase-2 (Line 40–42): When *HelpUpdate* enters *Phase-2* (Line 40–42), it means that all replicas it has traversed are already updated to *expected*. This thread is a successor that can clear the lock bit of G by pCASing G to *expected* (Line 41). Also, there might exist other successors that try to clear the lock bit of G . This is harmless, and *HelpUpdate* can directly return (Line 42).

Once the *HelpUpdate* returns, it means that all replicas must have already been updated to *expected* (but it is possible that G was updated again during the *HelpUpdate* or after the *HelpUpdate* returns). This is why *LoadPtr* can directly return the current value of the replica after calling *HelpUpdate* in *Case-2*, since this *Load* operation can be serialized before the *CASPtr* of G at this time. Due to the same reason, *CASPtr* can directly return true after calling *HelpUpdate* in *Phase-3*, since all replicas have already been updated to *new_val* at this time.

B.2 Linearizability

We use TLA+ to verify the linearizability of *CASPtr* and *LoadPtr*, and the code can be found in <https://anonymous.4open.science/r/spore-codebase-7625>.

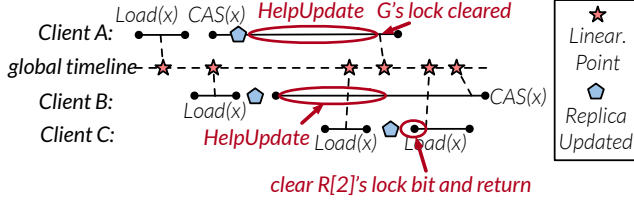


Fig. 5. Linearizability of G2

We also give a description of how linearizability is preserved. As shown in Fig. 5, the linearizable points (LP) of *CASPtr* and *LoadPtr* are all serialized on the global timeline:

(1) The LP of *CASPtr* is the point where all replicas are updated (Line 41 in Algorithm 1). For example, the LP of Client A's *CAS(x)* is the point where *HelpUpdate* finishes with all replicas updated to *new_val*, and *G*'s lock bit cleared.

(2) For reader starting with *old_val* (e.g., Client B's first *Load(x)* and Client C's first *Load(x)*), its LP is the time that the *pload* of a replica without lock bit set (line 3 in Algorithm 1). These readers are serialized before LP of the current *CASPtr* that updates *G* to *new_val*.

(3) For reader starting with *new_val* (e.g., Client C's second *Load(x)*), its LP is serialized after LP of the current *CASPtr* that updates *G* to *new_val*. Specifically, it is the time that all replicas are updated and the *G* is not changed again. Note that during the *LoadPtr*, the global pointer *G* can be unlocked by some thread that calls *HelpUpdate*, and changed again by another concurrent *CASPtr* (Client B's *CAS(x)* in this case). Fortunately, returning the value (Line 10 in Algorithm 1) will return the value that *LoadPtr* sees before *G* changes again.

(4) For concurrent *CASPtr* (i.e., another *CASPtr* issued before the current *CASPtr* finishes), it will see that *G* is locked, and first call *HelpUpdate* to help finish the last-round update of *G* (Line 14-15 in Algorithm 1). This concurrent *CASPtr*'s LP is the time that it successfully *CAS* the *G* to *new_val*, and finish updating all replicas by calling *HelpUpdate* (Line 20 in Algorithm 1).

B.3 Crash Consistency

Durable linearizability. Adding replicas does not compromise durable linearizability.

First, before using *CASPtr* and *LoadPtr*, the durable point (DP) of *G* is the time that it is successfully updated by a *pCAS*.

Second, after using *CASPtr* and *LoadPtr*, the DP is still the *pCAS* of the *G* (Line 17 in Algorithm 1): once the DP is reached (i.e., *G* is *pCAS*ed to *new_val*), replicas will eventually catch up with the *new_val*. Specifically, if any operation sees that its corresponding replica has the lock bit, it will enter the *HelpUpdate* phase to help update all replicas to the new *G*. If the operation that updates *G* with *pCAS* immediately crashes before setting any replica's value, the following *LoadPtr* will not see that new *G*, and will continue using the value in replicas. This will not cause any inconsistency for indexes.

LoadPtr operations using old replica values are linearized before this crashed *CASPtr*. Besides, the following *CASPtr* will see that *G* is locked, and call *HelpUpdate* to help update all replicas (Line 8 in Algorithm 1).

Fault isolation. *CASPtr* and *LoadPtr* will not block other concurrent operations given that they do not use locks or infinite loops. Thus, the fault isolation is also preserved.

B.4 Corner Cases

This subsection discusses how our algorithm handles corner cases. We show several concrete corner cases as follows.

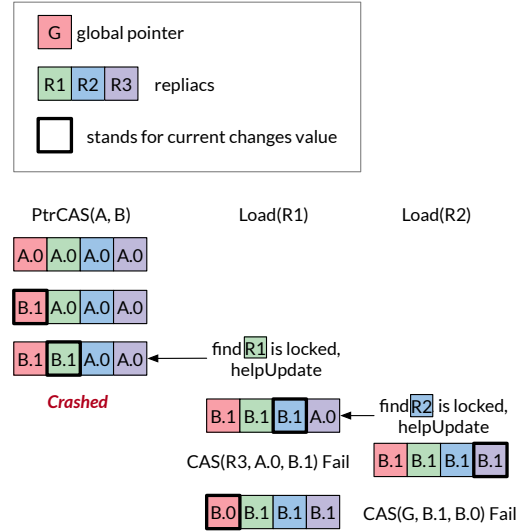


Fig. 6. Crashed *CASPtr*/*LoadPtr* before finish.

Case1: Crashed *CASPtr*/*LoadPtr* before finish (Fig. 6).

We design the *HelpUpdate* mechanism primarily to handle failures of *CASPtr* and *LoadPtr* operations that crash in the middle of surveying replicas. Without such failures, *CASPtr* could simply perform the replica update by itself.

HelpUpdate tolerates crashes during the survey phase because replica updates are cooperative: any operation can help propagate an observed update to the remaining replicas.

As shown in Fig. 6, if a *CASPtr* changes *G* from *old_val* = *A* to *new_val* = *B*, updates replica *R*₁, and then crashes, later operations that observe the modified replicas can continue the update on its behalf. In this case, *LoadPtr* on *R*₁ will see *R*₁ locked (*R*₁ = *B.1*, where .1 means the lock bit is set), and help update *R*₂ and *R*₃ to *B.1* by calling *HelpUpdate*.

If *CASPtr* fails immediately after updating *G*, concurrent *LoadPtr* operations that still read *R*_{*i*} = *A.0* can safely proceed with the old value and be linearized before that *CASPtr*. For example, in *ClevelHash*, *CASPtr* updates the context pointer when the hash table expands. If it updates the global context pointer and then crashes, other threads can still read the old context pointer and be linearized before the *CASPtr*. Reading from the old context does not cause any inconsistency in

the hash table. In BwTree, *CASPtr* updates the root node pointer when the root node is split or merged. If a thread crashes after updating the root node pointer, concurrent *Lookup* operations may miss the Δ node for a root split, root merge, root removal, insertion, or deletion, but this does not cause any inconsistency.

Concurrent *CASPtr* will see that G is locked, and call *HelpUpdate* to help update all replicas. The consistency of operations that overlap with a failed operation is ensured by the original designs of ClevelHash and BwTree.

Case2: Another CASPtr update G during HelpUpdate.

During the *HelpUpdate* phase, if another *CASPtr* updates G to a new value, the *HelpUpdate* can correctly handle this case, without updating replicas to incorrect value.

As shown in Fig. 7, *PtrCAS*(A, C) is issued after *PtrCAS*(A, B) is completed (i.e., G 's lock bit is cleared), and *LoadPtr*(R_1)'s *HelpUpdate* is still in progress.¹

Our *HelpUpdate* mechanism prevents *HelpUpdate* from overwriting R_i with an old value (e.g., if G changed from B to C by another *CASPtr*, we cannot update R_i from C to B). This overwrite problem can happen as a time-to-check-to-time-of-use (TCTOU) problem, i.e., *LoadPtr*(R_1) in Fig. 7 tries to help update R_2 , checks when $G = B.1$ and $R_2 = A.0$, but conducts the update when $G = C.1$ and $R_2 = C.1$. We avoid this by updating R_i with a CAS. Therefore, CAS R_2 from $A.0$ to $B.1$ will fail, avoiding the overwrite problem.

Note that updating $R_2 = B$ to $B.1$ is safe, because later *HelpUpdate* will check $G = C.1$ and $R_2 = B.1$ and update R_2 to $C.1$.

Case3: ABA problem (is prevented by epoch-based reclamation in our index cases). In the above case, we show that using CAS to update R_i can avoid the overwrite problem. However, CAS cannot avoid the ABA problem: as shown in Fig. 7, G is updated from A to B , and then updated again to A . In this case, R_2 can be updated to $A.0$ again, and the slow helper's CAS from $R_2 = A.0$ to $R_2 = B.1$ might succeed and leak an old value $B.1$.

Fortunately, the ABA problem is prevented by epoch-based reclamation in our index cases. Updating a pointer to a previously used node can only happen when this node is freed and reallocated (i.e., A is a pointer to node A , node A is freed and reallocated, and the index reuses this node). Since *LoadPtr*(R_1) has not finished its operation in this case, this node is still accessible by *LoadPtr*(R_1) and thus will not be reclaimed by the epoch-based garbage collection mechanism.

Nodes are not removed from the index unchanged and then reinserted; BwTree always allocates a new Δ node for each update. ClevelHash also always allocates a new context or key/value pair for each update.

References

- [1] Robert Binna, Eva Zangerle, Martin Pichl, Günther Specht, and Viktor Leis. Hot: A height optimized trie index for main-memory database systems. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18*, pages 521–534, New York, NY, USA, 2018. Association for Computing Machinery.
- [2] Tudor David, R. Guerraoui, and Vasileios Trigonakis. Asynchronized concurrency: The secret to scaling concurrent search data structures. *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2015.
- [3] Se Kwon Lee, Jayashree Mohan, Sanidhya Kashyap, Taesoo Kim, and Vijay Chidambaram. Recipe: converting concurrent dram indexes to persistent-memory indexes. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP '19*, pages 462–477, New York, NY, USA, 2019. Association for Computing Machinery.
- [4] Viktor Leis, Florian Scheibner, Alfons Kemper, and Thomas Neumann. The art of practical synchronization. In *Proceedings of the 12th International Workshop on Data Management on New Hardware*, pages 1–8. Association for Computing Machinery.
- [5] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. Cache craftiness for fast multicore key-value storage. In *Proceedings of the 7th ACM European Conference on Computer Systems*, pages 183–196. Association for Computing Machinery.
- [6] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. Lupin: Tolerating partial failures in a cxl pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems, DIMES '24*, pages 41–50, New York, NY, USA, 2024. Association for Computing Machinery.

¹Note that due to infrequent updates of replicated pointers, the case of concurrent *CASPtr* is rare in practice.

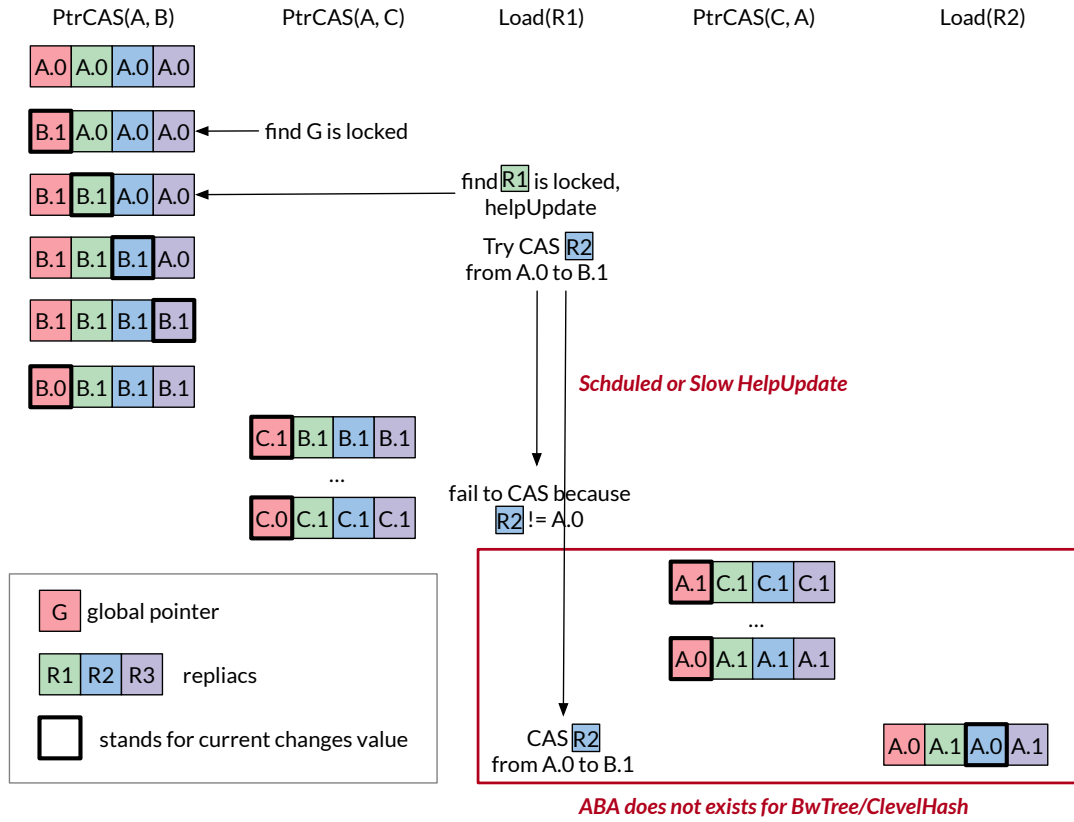


Fig. 7. Another CASPtr update G during *HelpUpdate*, and the ABA problem.